Intro to AI

Project - 1

This Ship is on Fiiiiire!

Data, Analysis, and Write-up

By:

- Raj Patel: rap378
- Yug Patel: yp517

1. Introduction

In this project, a bot on a burning ship has to reach a button that puts out the fire before the fire reaches it. The fire spreads faster when q (the ship's flammability) is higher, so the question is how the bot should decide where to go.

We compared four bots. Bot 1 plans a shortest path once and ignores the fire, Bot 2 replans every step around the fire, and Bot 3 also stays away from cells next to the fire. Bot 4 is our own, it gives every cell a cost based on how risky it is and takes the cheapest path to the button.

We ran all four bots on the same random ships and starting positions, with 1,000 trials for every value of q to measure success rates, plus a separate 100-trial run to see why each bot failed. We found that all four bots do about equally well when q is very low and almost equally badly when q is above 0.6. Bot 1 does the worst because it never reacts to the fire, and Bot 4 does slightly but consistently better than the others between about q = 0.2 and 0.55.

2. Our Approach and Key Decisions

We built the project one piece at a time and tested each part by printing the grid and debug messages in the terminal and watching the bot move step by step. Here are the choices the handout left up to us and the problems we ran into.

From a greedy bot to BFS. Our first bot just stepped toward the button and backtracked when it hit a wall. It kept getting stuck whenever it had to move away from the button to get around a wall, so we switched to BFS. Since Bots 1, 2, and 3 only differ in when they plan and

which cells they can step on, we used one BFS function for all three, with an option for Bot 3's extra rule.

No path means failure. If Bot 2, 3, or 4 cannot find any path around the fire, we end the run. The fire never goes out, so if there is no path now, there never will be.

Fire reaching the button. The handout does not cover this, but we count it as a failure, since pressing the button would mean stepping into the fire.

Making the comparison fair. Every trial uses one ship and one set of starting positions for all four bots, so no bot gets an easier ship. Each bot still gets its own random fire, which is one reason we needed so many trials.

Bugs we caught. Our first dead-end removal did not actually open any cells, and it stopped after a fixed number of tries. We also forgot to clear the bot's starting cell on the grid, so the fire could never burn it and Bots 2 and 3 could not plan a path back through it. We fixed both before collecting any results.

3. Bot 4 Design

Where the Idea Came From:

When we thought about how Bots 2 and 3 can fail, we saw they have opposite weaknesses. Bot 2 only avoids cells that are already burning, so it will happily walk right next to the fire, and the fire can spread onto it on the very next step. Bot 3 refuses to step next to the fire at all, so if the only "safe" route is a long detour, it takes it, even if the fire reaches the button while it is walking around. Neither bot ever asks how risky this cell is compared to how much longer the safer route is.

So our idea for Bot 4 was to stop treating cells as just allowed or not allowed, and give every cell a cost instead. Safe cells are cheap, risky cells are expensive, and the bot picks the path with the lowest total cost. This way, it can take a short detour around the fire when one exists, but it will not take a huge detour just to avoid one risky cell.

How Bot 4 Decides Each Step:

Every time step, Bot 4 does three things:

1. It runs one BFS starting from all burning cells to find how far each open cell is from the fire, moving only through hallways, not through walls.
2. Give every open cell a cost:

   cost = 1 + 5 × fire risk + 5 / (fire distance + 1)

   - The 1 is the normal cost of a step, so shorter paths are still preferred.
   - The **fire risk** is the chance the cell catches fire on the next step, 1 − (1 − q)^K. It is only non-zero for cells touching the fire, and it grows with q.
   - The distance penalty makes cells near the fire more expensive and fades out as the fire gets farther away (Table 1).
3. It runs Dijkstra to find the cheapest path to the button, takes the first step, and repeats on the next step. We needed Dijkstra instead of BFS because cells now have different costs. Burning cells are never allowed.

Distance from fire (steps)	1	2	3	5	10	20
Penalty	2.50	1.67	1.25	0.83	0.45	0.24

Table 1: The distance penalty for different distances from the nearest fire.

How we got this formula. We started from the fire-risk term because it comes straight from the fire formula. But it only looks one step ahead: a cell two steps from the fire has zero risk, even if it could be burning by the time the bot gets there. So we added the distance penalty. Then we noticed the risk term barely mattered, since the probability is never more than 1 while the distance penalty right next to the fire is 2.5. So we multiplied it by 5 as well. With these weights, walking right past the fire adds about 10 to the cost at q = 0.3, compared to about 1 to 2 for the same number of cells far from the fire. So Bot 4 will take a detour of about 5 or 6 extra steps to avoid the fire (about 7 or 8 at q = 0.8), but not a longer one. We did not adjust these numbers carefully, and Section 5 looks at whether they were good choices.

A mistake we fixed. At first, our distance BFS also went through walls, so a fire behind a wall looked close even when it was 20 steps away by hallway. We changed it to move only via open cells.

Why Is This Not Just a Wider Buffer?

The handout says Bot 4 cannot be Bot 3 with a bigger buffer, so here is how it is different:

- Bot 3 bans cells next to the fire unless it has no other choice. Bot 4 never bans anything except cells that are actually burning. It compares how risky a route is and how much longer the option is, so the same cell near the fire might be avoided in one situation and used in another.
- It uses q. Bot 3 behaves the same at every q. Bot 4's risk term depends on q, and because we weight it by 5, a cell next to the fire gets much more expensive as the ship gets more

flammable (from 0.5 at q = 0.1 to 4 at q = 0.8). So Bot 4 is willing to take longer detours when the fire spreads faster.

- It looks at hallway distance. A fire on the other side of a wall does not worry Bot 4, but a fire a few steps down the same hallway does.

Making the Code Efficient

- Ship generation keeps a list of candidate cells instead of rescanning the whole grid, and dead-end removal only rechecks the cells around the one it just opened.
- Fire spread only checks cells touching the fire, once each per step.
- Bot 4 gets the fire distances for all cells from one BFS per step.
- BFS and Dijkstra never expand the same cell twice.
- Each ship is generated once and reused for all four bots.

With all of this, our full run at D = 30 (17 values of q × 1,000 trials × 4 bots) took about 7.5 minutes on our device. One thing we would still improve: both searches store a full copy of the path for every cell they visit. Storing just the previous cell and rebuilding the path at the end would make the searches faster on bigger ships.

4. Results

How We Ran the Experiments

For every trial, we generated a new ship, placed the bot, button, and fire, and ran all four bots on it. Our first runs used q = 0, 0.1, …, 1.0. Those showed that the bots only really differ below about q = 0.6, so we added extra points every 0.05 in that range, for 17 values of q in total.

We also started with 100 trials per q, but the curves jumped around too much (Figure 2). Lines crossed back and forth, and some bots even did better at a higher q than at a lower one, which can only be noise. So our main results use 1,000 trials per q on a 30×30 ship. Bigger ships were much slower to run, so we used D = 50 with 100 trials only to check that the same pattern shows up on a bigger ship (Figure 3), and it does.

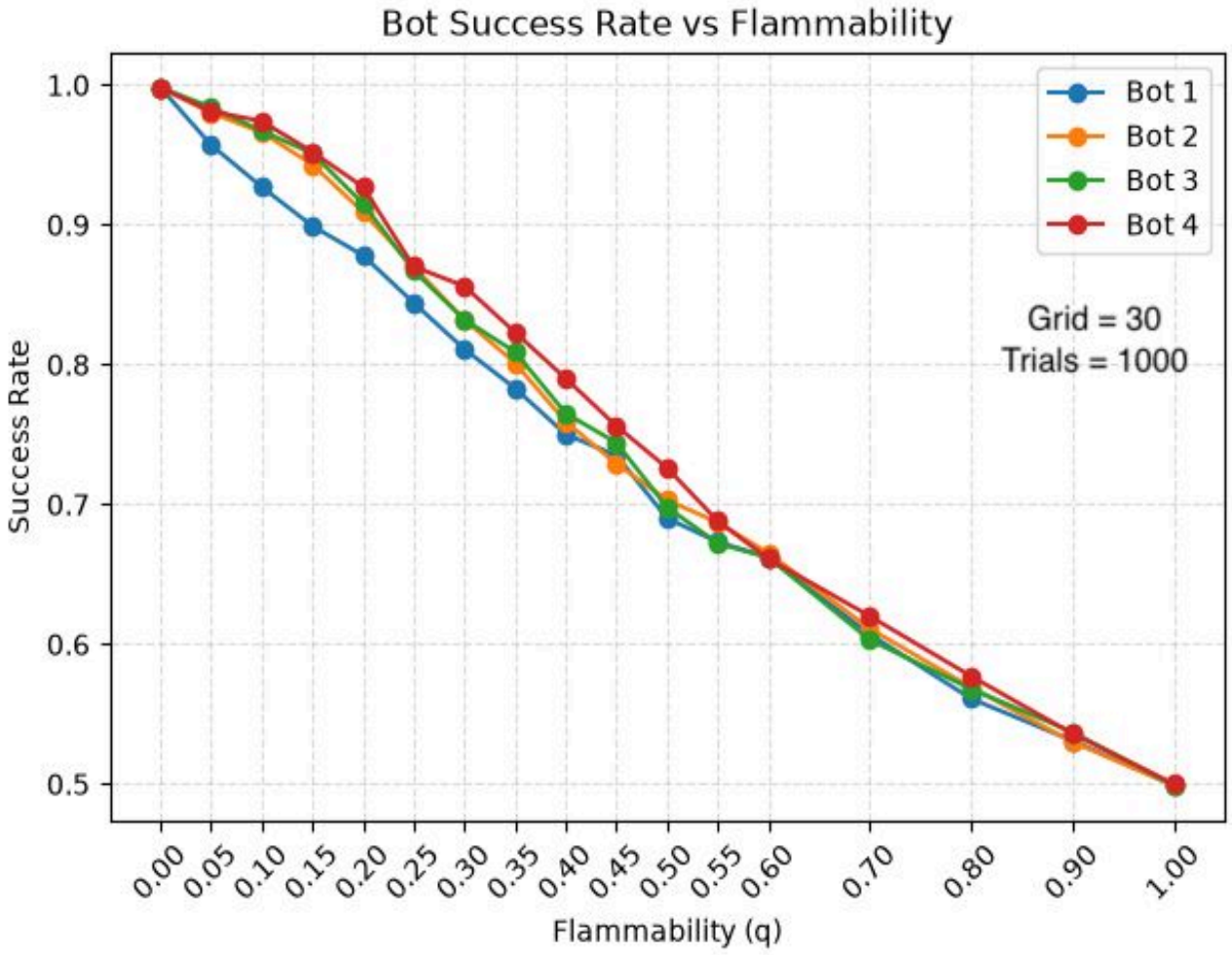


Figure 1: Success rate vs. flammability q for all four bots (D = 30, 1,000 trials per q).

What the Graph Shows

Figure 1 splits into three regions:

- Very low q (0 to 0.05): all bots are almost perfect. The fire barely spreads, so almost any path works. Even at q = 0, the bots succeed about 99.7% of the time, not 100%. These failures are cases where the starting fire sits on the only route to the button, so there is no path from the beginning.
- Middle q (about 0.05 to 0.55): the bots differ. Bot 1 is clearly the worst here, about 3 to 5 points below the others (about 0.90 vs. 0.95 at q = 0.15), because it never reacts to the fire. Bots 2 and 3 are almost identical, which surprised us, since Bot 3's extra caution barely seems to change anything. We look at why in Section 5. Bot 4 is on top for most of this range.
- High q (0.6 and above): all bots are similar. The fire spreads so fast that planning does not help, and it mostly comes down to whether the bot started closer to the button than the fire. At q = 1, every bot succeeds about 50% of the time. That makes sense, at q = 1, the fire moves one cell in every direction each step, just like the bot, and since both start in random spots, each wins the race about half the time.

Where Bot 4 Does Better

Bot 4 does best between about q = 0.2 and 0.55. It is above or tied with Bots 2 and 3 at every q in that range, usually by 2 to 3 points (about 0.86 vs. 0.83 at q = 0.3, and 0.79 vs. 0.76 at q = 0.4). These gaps are small. With 1,000 trials, a single point can be off by about 2 to 3 points just from luck, so one point on its own does not prove much. But Bot 4 comes out ahead at almost every q in this range, and luck would not push it the same way many times in a row.

Outside this range, Bot 4 is tied with the others, which is what we expected: at low q there is nothing to avoid, and at high q there is no time to avoid it.

This was not true for our first version of Bot 4, before we multiplied the fire risk by 5. Its curve sat almost on top of Bot 2's, which told us it was mostly making the same moves as Bot 2. Making the risk term bigger is what got Bot 4 to actually choose different routes.

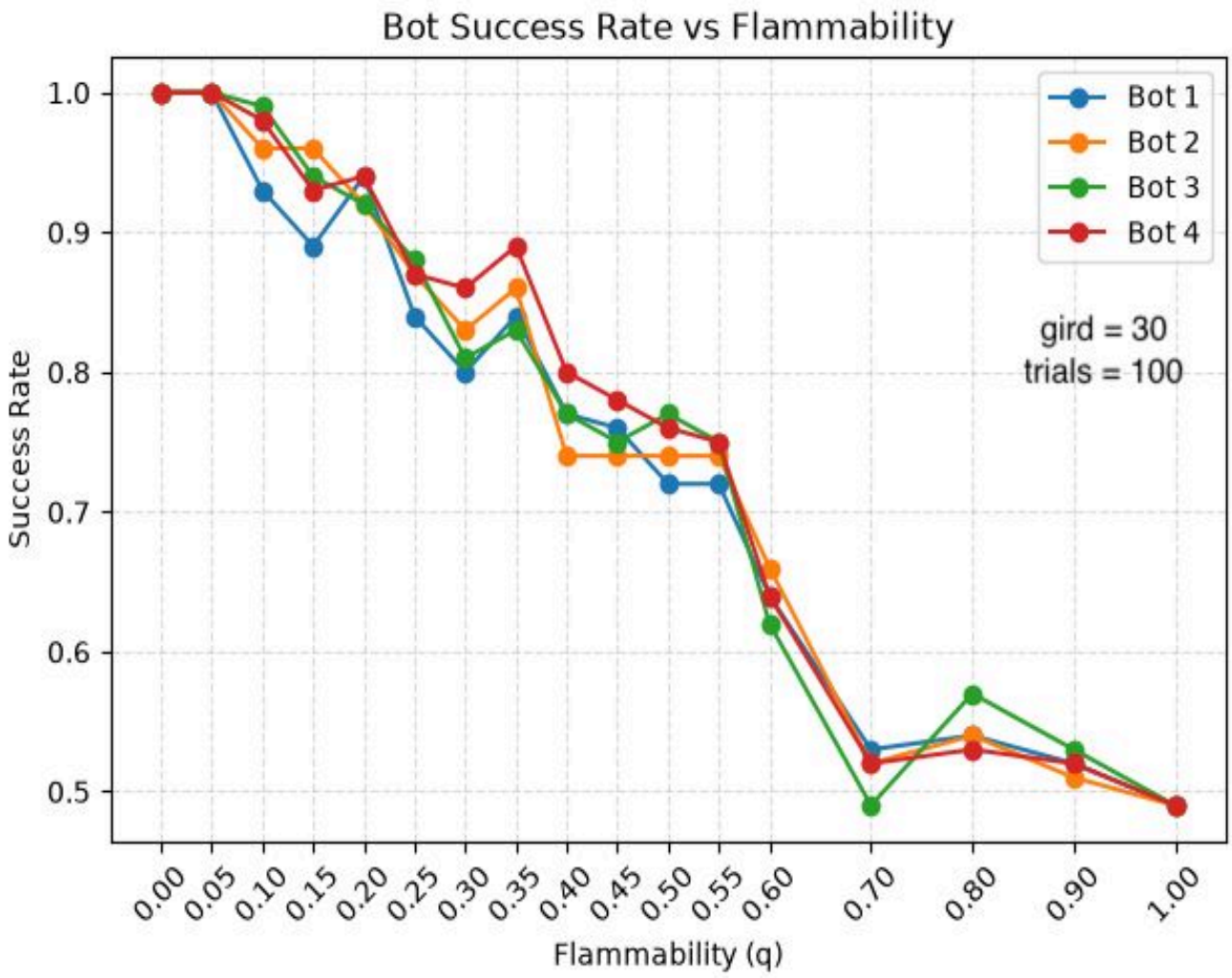


Figure 2: The same experiment with only 100 trials per q (D = 30). The lines cross back and forth because of noise.

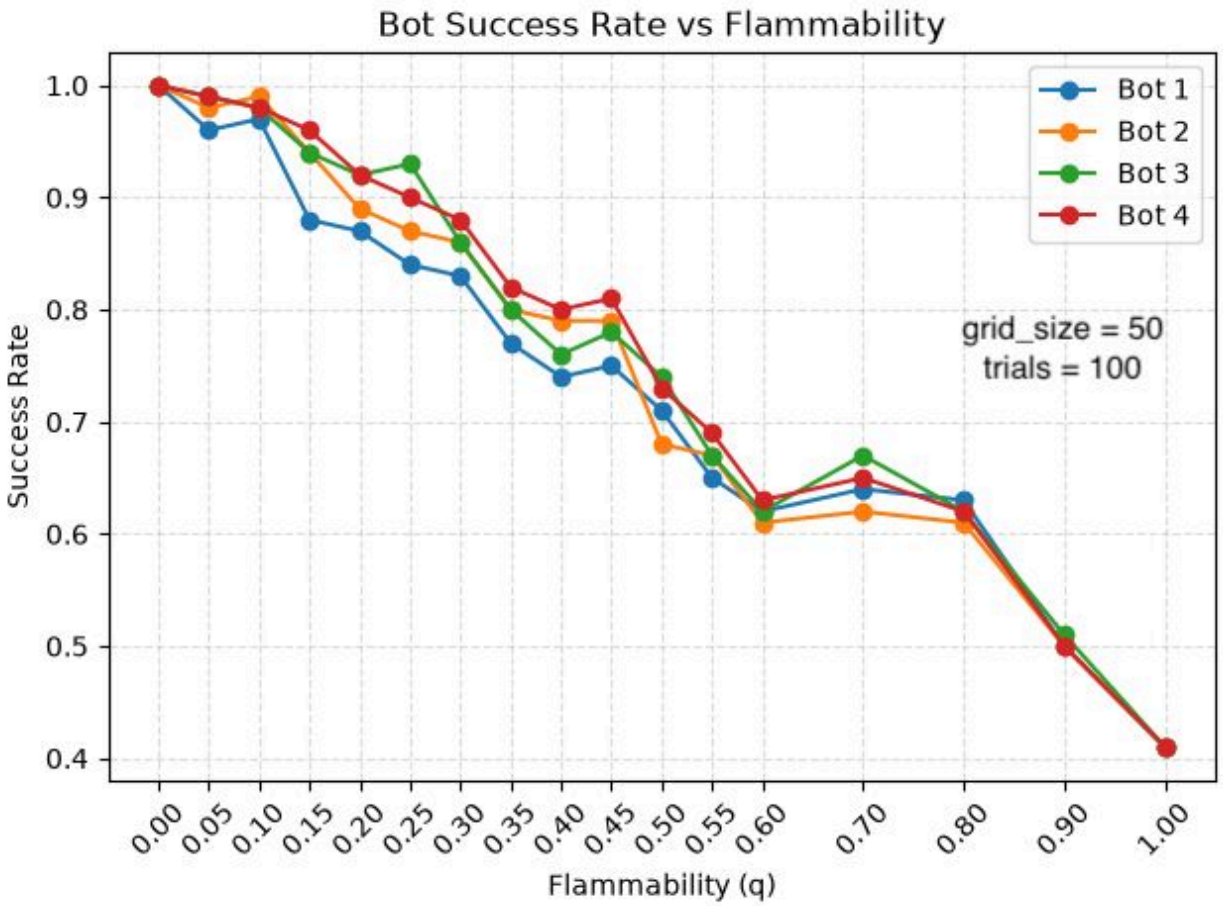


Figure 3: A bigger ship (D = 50, 100 trials per q). Noisy, but the same overall shape as Figure 1.

5. Why the Bots Fail

To see how each bot fails, we recorded the reason for every failure (Figure 4). There are four ways to fail: the bot walks into a burning cell, the fire spreads onto the bot, the fire reaches the button first, or no fire-free path is left.

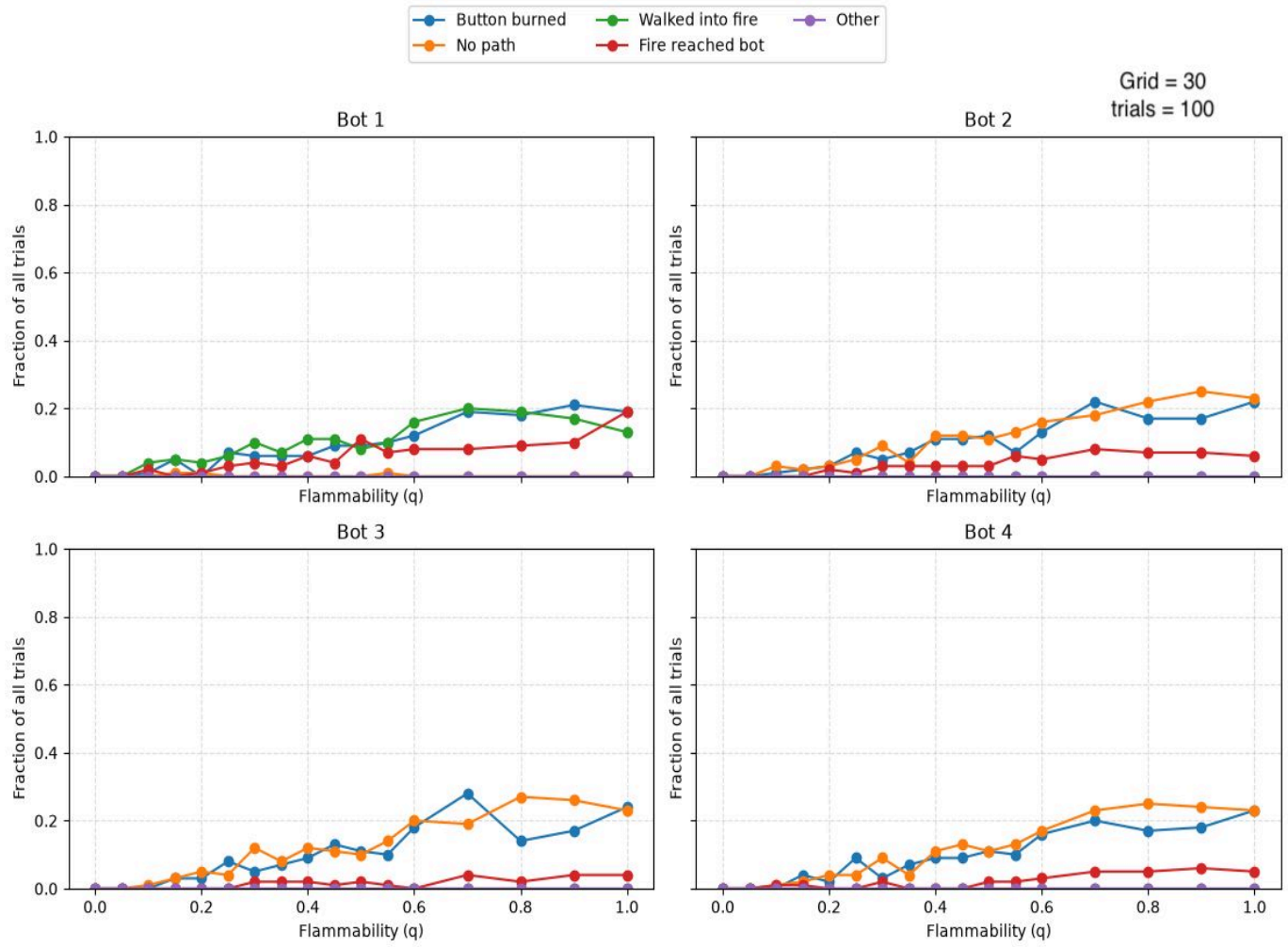


Figure 4: Why each bot fails, as a fraction of all trials (D = 30, 100 trials per q).

Bot 1 is the only bot that walks into fire. Since it never looks at the fire after planning, it keeps following its path even after the fire has spread onto it, and for most of the middle range this is one of its biggest causes of failure. This is clearly avoidable: Bots 2, 3, and 4 never walk into fire, just because they replan every step. That is probably most of the 3 to 5 point gap between Bot 1 and the others in Figure 1.

Bots 2, 3, and 4 mostly fail because the fire cuts off every path or reaches the button first. Many of these failures cannot be avoided, especially at high q. If the fire starts closer to the button than the bot does, which is why all four bots end up around 50% at q = 1.

Why Bots 2 and 3 end up the same. Figure 4 helps answer the question from Section 4. Our 100-trial failure data is noisy, but Bot 3 has fewer "fire reached bot" failures than Bot 2, so its extra caution does what it is supposed to, it keeps a gap from the fire. But those trials do not turn into wins. They tend to show up as "button burned" or "no path" instead, because while Bot 3 takes the longer, safer route, the fire has more time to reach the button or block the way. The two effects roughly cancel out, which is why their curves in Figure 1 overlap.

Bot 4 and our weights. Bot 4's "fire reached bot" failures fall somewhere between Bot 2's and Bot 3's. It stays away from the fire more than Bot 2 does, but without Bot 3's long detours, which fits its small lead in Figure 1. Looking at its failures also showed us some weak spots in our design:

- It only reacts to where the fire is now. It does not think about where the fire will be or which routes are about to close. When a narrow hallway is the only way to the button, Bot 4 does not hurry through it before the fire blocks it, so some of its "no path" failures might have been avoided by going through sooner.
- The distance penalty does not change with q. At high q, a 7 or 8 step detour gives the fire 7 or 8 extra steps to reach the button, which can be worse than just walking past the fire quickly.
- It never looks at how close the fire is to the button. If the fire is about to reach the button, the right move is to rush, not to take a detour.

- We only tried two values for the risk weight, 1 and 5. Going from 1 to 5 is what made Bot 4 behave differently from Bot 2, so trying more values, or making the weights depend on q, is probably the easiest way to improve it.

Could a better decision have saved them? For Bot 1, yes, in many cases, just replanning would have kept it out of the fire. For Bots 2 to 4, it depends. When the fire starts between the bot and the button, or closer to the button, nothing would have helped. But in some cases, rushing through a hallway before the fire closes it, or not wasting steps on a detour while the fire heads for the button, could have saved the bot. None of our bots think about where the fire will be in the future, which is what we look at next.

6. The Ideal Bot

What it would know. The ideal bot would use the same information our bots already have: the ship layout, where the fire is, and q. The difference is that it would use them to predict the future instead of only reacting to the present.

What it would compute. Since we know exactly how the fire spreads, the bot could simulate it. Starting from the current fire, it would run the fire forward a few hundred times, each time with different random rolls. Each simulation is one possible future. The bot would then take a few candidate routes (the routes Bots 2, 3, and 4 would pick) and check how many of those futures each route reaches the button before the fire does, then take the route that works most often. This fixes the weak spots we found in Section 5. The bot would see which hallways are about to close and whether the fire is heading for the button, so it would know when to hurry and

when to take a detour. In a way, our Bot 4 is a very cheap version of this. Its risk term looks one step ahead, and its distance penalty is a rough guess about everything after that.

Computation vs. time. This is expensive: hundreds of fire simulations over the whole ship, every single step. In our simulation, the fire waits for the bot to finish thinking, but in a real fire it would not. So the bot should only think hard when the decision actually matters. When the fire is far away, or q is low, all the routes are about equally safe (all our bots agree there, as Figure 1 shows), so it should just take the shortest path. It should only spend the extra time when the fire is close, and the routes really differ. It could also save time by reusing the last step's simulations instead of starting over, and by running fewer simulations when time is short.

When to just make a break for it. We think the bot should stop thinking and run in four cases:

- It is clearly closer to the button than the fire is. Any detour just gives the fire more time.
- q is high. Figure 1 shows that above about q = 0.6, planning does not help, because every step counts.
- Every route passes near the fire. The shortest one spends the least time in danger.
- The only hallway to the button is about to close. Rushing through it beats looking for a way around.

We also never let our bots stay in place, even though the handout allows it. Since the fire only grows, waiting can never open a new path, so we think it would rarely help.

7. Conclusion

The biggest lesson from this project is that replanning matters most. Just reacting to the fire every step (Bots 2 to 4) beat planning once (Bot 1) by 3 to 5 points in the middle range of q. Being more careful was not automatically better. Bot 3 stays farther from the fire, but its longer routes give the fire time to reach the button, so it ended up tied with Bot 2. Bot 4's balance between risk and path length gave a small but steady lead between q = 0.2 and 0.55, and its weights mattered a lot. Our first version acted almost exactly like Bot 2 until we made the risk term five times bigger. Above q = 0.6, nothing we tried helped much, because the fire is simply too fast.

A few things surprised us along the way: Bot 3 not doing better than Bot 2, our first Bot 4 copying Bot 2, and how noisy 100 trials turned out to be. If we had more time, we would adjust Bot 4's weights properly and try predicting the fire's future spread, like the ideal bot above.